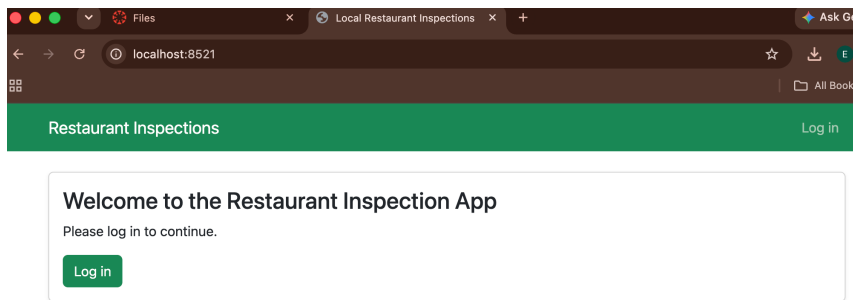


SID4: 1421
PORT_BASE: 8521
PREFIX: s1421
SEED: 1421
VERIFY_SEED: 2614221
DOMAIN_ID: 5
Hardware: Apple M2 Pro, 16GB
Local model used: sentence-transformers/all-MiniLM-L6-v2
Git link: <https://github.com/ericezhao/data260-1421>
Tag link:
Tagged commit hash:

Part 1 – Routers & Session Management

1. index page:



```
14 VALID_USERNAME = "admin"
15 VALID_PASSWORD = "password"
16 IDLE_SECONDS = 120 # impletment session timeout, set to 2 minutes
17
18
19 def logged_in(request: Request):
20     # Check if the user is logged in and within the idle timeout
21     user = request.session.get("user")
22     if not user:
23         return None
24     last_active = request.session.get("last_active")
25     now = int(time.time())
26     if last_active is None or now - int(last_active) > IDLE_SECONDS:
27         request.session.clear()
28         return None
29     request.session["last_active"] = now # update the last active time
30     return user
31
32
33 @router.get("/") # route to the home page
34 def home(request: Request):
35     user = logged_in(request)
36     return templates.TemplateResponse(request, "index.html", {"user": user})
```

2. login page:

A screenshot of a web browser showing the login page for 'Restaurant Inspections'. The browser's address bar shows 'localhost:8521/login'. The page has a green header with the text 'Restaurant Inspections' and a 'Log in' button. Below the header is a white box titled 'Inspector login' containing two input fields: 'Username' with the value 'admin' and 'Password' with masked characters '.....'. A green 'Log in' button is at the bottom of the box.

wrong password:

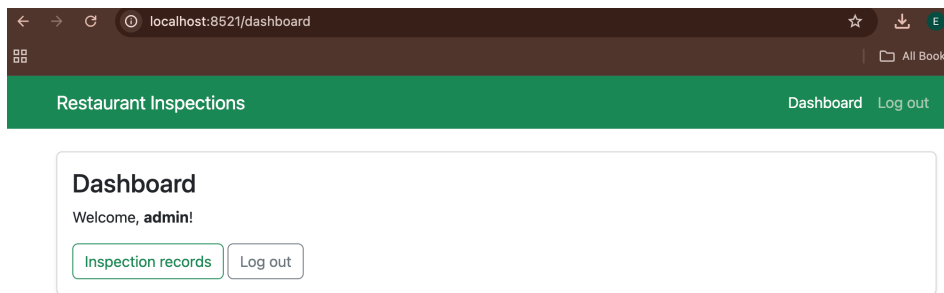
A screenshot of the same login page, but with an error message. A pink box above the input fields contains the text 'Invalid username or password.' The 'Username' field is empty, and the 'Password' field is also empty. The 'Log in' button remains at the bottom.

```
@router.get("/login")
def login_page(request: Request):
    error = request.session.pop("login_error", None) # pop the login error from the session
    return templates.TemplateResponse(
        request,
        "login.html",
        {"user": logged_in(request), "error": error},
    )

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

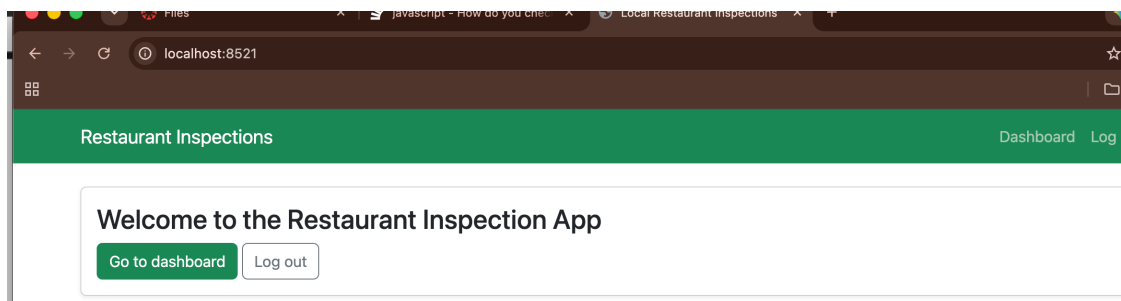
    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        request.session["user"] = username # store the username in the session
        request.session["last_active"] = int(time.time()) # set the last active time to now
        request.session.pop("login_error", None) # pop the login error from the session if it exists
        return RedirectResponse(url="/dashboard", status_code=HTTP_302_FOUND)
    # If the credentials are invalid, store the error in the session and redirect to the login page
    request.session["login_error"] = "Invalid username or password."
    return RedirectResponse(url="/login", status_code=HTTP_302_FOUND)
```

3. dashboard page:

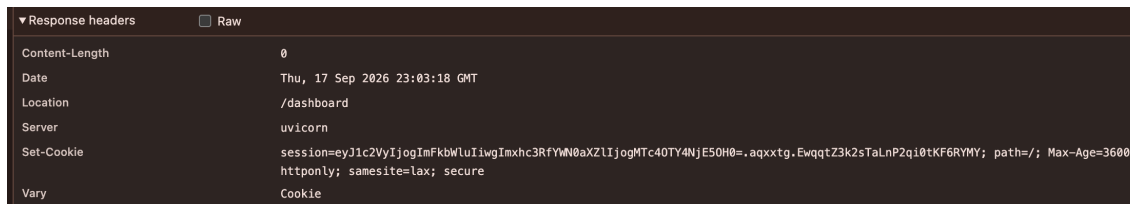


```
@router.get("/dashboard")
def dashboard(request: Request):
    user = logged_in(request)
    if not user:
        return RedirectResponse(url="/login", status_code=HTTP_302_FOUND)
    return templates.TemplateResponse(request, "dashboard.html", {"user": user})
```

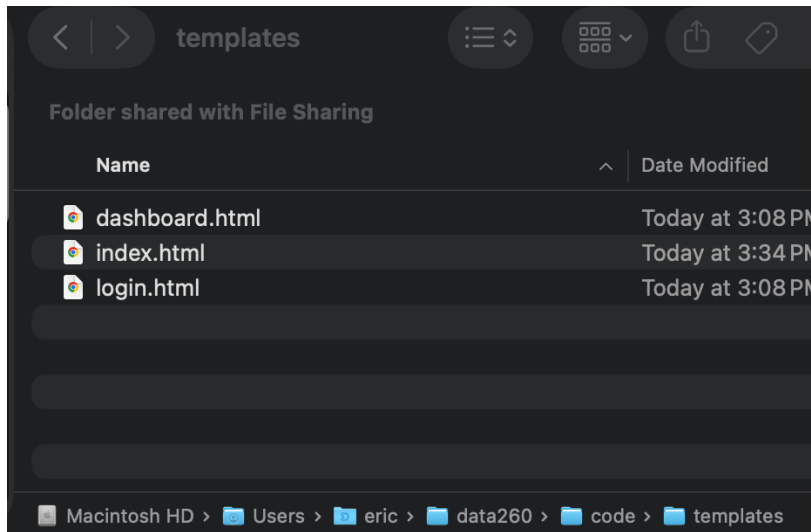
4. Dashboard is accessible from index page after successful login and there is a log out button:



5. Set-Cookie response header:



6. Template directory:



7. 120 seconds idle session time out works with 302 message:

```
echo "after idle:"
[curl -b /tmp/idle.txt -s -o /dev/null -D - http://localhost:8521/dashboard | he]
ad -15
right after login:
HTTP/1.1 200 OK
date: Thu, 17 Sep 2026 23:13:06 GMT
server: uvicorn
content-length: 1129
content-type: text/html; charset=utf-8
vary: Cookie
set-cookie: session=eyJlc2VyIjogImFkbWluIiwgImxhc3RfYWN0aXZlIjogMTc4OTY4Njc4N30=
.aqx0Aw.G_XU1lRXj0q8x6I6JK3fM8PX46E; path=/; Max-Age=3600; httponly; samesite=la
x; secure

waiting 125 seconds...
after idle:
HTTP/1.1 302 Found
date: Thu, 17 Sep 2026 23:15:11 GMT
server: uvicorn
content-length: 0
location: /login
vary: Cookie
set-cookie: session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponl
y; samesite=lax; secure
```

Timer is built into `logged_in` function:

```
def logged_in(request: Request):
    #Check if the user is logged in and within the idle timeout
    user = request.session.get("user")
    if not user:
        return None
    last_active = request.session.get("last_active")
    now = int(time.time())
    if last_active is None or now - int(last_active) > IDLE_SECONDS:
        request.session.clear()
        return None
    request.session["last_active"] = now # update the last active time
    return user
```

8. auth.py full code:

```
from pathlib import Path # to locate the templates directory
import time # to track the last active time of the user

from fastapi import APIRouter, Request, Form # to handle the request and form data
from fastapi.responses import RedirectResponse # to redirect the user to other pages
from fastapi.templating import Jinja2Templates # to render the templates
from starlette.status import HTTP_302_FOUND # to return the HTTP status code

router = APIRouter() #create a router object
templates = Jinja2Templates(directory=str(Path(__file__).resolve().parent /
"templates"))

# Hardcoded credentials for demo purposes only
# In real applications, credentials come from a database
VALID_USERNAME = "admin"
VALID_PASSWORD = "password"
IDLE_SECONDS = 120 # impletment session timeout, set to 2 minutes

def logged_in(request: Request):
    # Check if the user is logged in and within the idle timeout
    user = request.session.get("user")
    if not user:
        return None
    last_active = request.session.get("last_active")
    now = int(time.time())
    if last_active is None or now - int(last_active) > IDLE_SECONDS:
        request.session.clear()
        return None
    request.session["last_active"] = now # update the last active time
    return user

@router.get("/") # route to the home page
def home(request: Request):
    user = logged_in(request)
    return templates.TemplateResponse(request, "index.html", {"user": user})

@router.get("/login")
def login_page(request: Request):
    user = logged_in(request)
    error = request.session.pop("login_error", None) # pop the login error from the
session if it exists
    return templates.TemplateResponse(
```

```
        request, "login.html", {"user": user, "error": error}
    )

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        request.session["user"] = username # store the username in the session
        request.session["last_active"] = int(time.time()) # set the last active time
        to now
        request.session.pop("login_error", None) # pop the login error from the
        session if it exists
        return RedirectResponse(url="/dashboard", status_code=HTTP_302_FOUND)
    # If the credentials are invalid, store the error in the session and redirect to
    the login page
    request.session["login_error"] = "Invalid username or password."
    return RedirectResponse(url="/login", status_code=HTTP_302_FOUND)

@router.get("/dashboard")
def dashboard(request: Request):
    user = logged_in(request)
    if not user:
        return RedirectResponse(url="/login", status_code=HTTP_302_FOUND)
    return templates.TemplateResponse(request, "dashboard.html", {"user": user})

@router.get("/logout")
def logout(request: Request):
    request.session.clear()
    return RedirectResponse(url="/", status_code=HTTP_302_FOUND)
```

Part 2 - Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG)

Chunking:

```

52 # Chunking -> Nodes
53 def make_nodes(docs, technique, embed_model): # create nodes from the documents
54     parsers = {
55         "token-based chunking": TokenTextSplitter(chunk_size=512, chunk_overlap=50),
56         "semantic chunking": SemanticSplitterNodeParser(embed_model=embed_model, buffer_size=1),
57         "sentence-window chunking": SentenceWindowNodeParser.from_defaults(window_size=3),
58     }
59     return parsers[technique].get_nodes_from_documents(docs)

```

Indexing:

```

62 # Indexing for fast similarity search using faiss
63 def make_index(nodes, embed_model):
64     dim = len(embed_model.get_text_embedding("dummy text")) # get the dimension of the embedding vector
65     storage = StorageContext.from_defaults(
66         vector_store=FaissVectorStore(faiss_index=faiss.IndexFlatIP(dim))
67     ) # custom storage context using faiss for similarity search
68     return VectorStoreIndex(nodes, storage_context=storage, embed_model=embed_model) # nodes are tokens
69

```

Retrieval:

```

# Retrieval
def retrieve(index, embed_model, query, k):
    qvec = np.asarray(embed_model.get_query_embedding(query), dtype=float) # generate query vector
    t0 = time.perf_counter() # start time
    hits = index.as_retriever(similarity_top_k=k).retrieve(query) # retrieve the top k hits
    latency_ms = (time.perf_counter() - t0) * 1000 # calculate the latency in milliseconds
    rows = [] #
    doc_vecs = []
    for rank, hit in enumerate[Any](hits, 1): # enumerate the hits and get the rank, hit
        text = (hit.node.metadata or {}).get("window") or hit.node.get_content() # select the text from the hit
        dvec = np.asarray(embed_model.get_text_embedding(text), dtype=float) # generate document vector
        doc_vecs.append(dvec)
        rows.append(
            {
                "rank": rank,
                "store_score": None if hit.score is None else float(hit.score),
                "cosine_sim": float(cosine_similarity([qvec], [dvec])[0, 0]),
                "chunk_len": len(text),
                "preview": " ".join(text.split()[:160]),
                "source": (hit.node.metadata or {}).get("file_name", ""),
            }
        )

    return {
        "query_dim": int(qvec.size),
        "query_first8": [round(float(x), 4) for x in qvec[:8]],
        "latency_ms": round(latency_ms, 2),
        "hits": rows,
        "query_shape": list[Any](qvec.shape),
        "docs_shape": list[Any](np.vstack(doc_vecs).shape)
    }

```

Outputs

Embedding:

```
Query embedding (shared query q4)

=== token-based chunking | q4 ===
query dim 384 first8 [-0.0033, -0.0123, -0.0354, -0.0094, 0.078, 0.0422, 0.041, 0.0106]
query_shape [384] docs_shape [5, 384]

Same query embedding for semantic and sentence-window (identical first8).
```

Token-based:

```
=== token-based chunking | q4 ===
query dim 384 first8 [-0.0033, -0.0123, -0.0354, -0.0094, 0.078, 0.0422, 0.041, 0.0106]
query_shape [384] docs_shape [5, 384]
latency_ms 5.66

rank store_score cosine_sim chunk_len preview
1 0.699368 0.600380 2274 food-contact surfaces and ultimately in the food being prepared or served. 7-206.13 Tracking Powders, Pest Control and Monitoring. The use of tracking powder
2 0.696320 0.664152 1958 82. Effective food-contact notifications are listed at http://www.accessdata.fda.gov/scripts/fcn/fcnNavigation.cfm?rpt=fcsListing&displayAll=false&page=17, and
3 0.686384 0.621786 1854 Storage Containers Identified with Common Name of Food (C) 3-305.13 Vended Time/Temperature Control for Safety Food, Original Container (C) 3-601.11 Standards o
4 0.670413 0.581814 1808 and maintenance of a FOOD ESTABLISHMENT, such as for the cleaning and SANITIZING of EQUIPMENT and UTENSILS and the control of insects and rodents, shall be all
5 0.639708 0.586980 1755 and Enforcement No Changes. Annex 2. References 1-201.10 Amended to revise § 1201.10 to update titles, web links and dates for the following references: #3 I
```

Semantic:

```
=== semantic chunking | q4 ===
```

query dim 384 first8 [-0.0033, -0.0123, -0.0354, -0.0094, 0.078, 0.0422, 0.041, 0.0106]
query_shape [384] docs_shape [5, 384]
latency_ms 6.43

rank	store_score	cosine_sim	chunk_len	preview
1	0.635298	0.372099	191	17, No. 1, in: http://wwwnc.cdc.gov/eid/article/17/1/p2-1101_article.htm 21. The Public Health Service Act, 42 U.S.C. Section 243. General Grant of Authority f
2	0.621964	0.607355	1655	97:258-260. 2. Code of Federal Regulations, Title 9, Subpart C, Section 424.21(b) Food ingredients and sources of radiation. 3. Code of Federal Regulations, Tit
3	0.609795	0.580247	1441	2006. Interactions Affecting the Proliferation and Control of Human Pathogens on Edible Plants. J. Food Sci. 71(8), R89 – R99. FDA Food Code 2022 Annex 2. Refer
4	0.598549	0.581428	3910	(c) The DEPARTMENT shall adopt regulations to govern the notice required by this section in order to effectuate the purposes of this section. 114094. Menu label
5	0.594480	0.505935	33368	This record should not be relied upon as an absolute comparison that identifies each and every change. General Numerous editing changes were made throughout the

Sentence-window:

```
=== sentence-window chunking | q4 ===
```

query dim 384 first8 [-0.0033, -0.0123, -0.0354, -0.0094, 0.078, 0.0422, 0.041, 0.0106]
query_shape [384] docs_shape [5, 384]
latency_ms 6.58

rank	store_score	cosine_sim	chunk_len	preview
1	0.863730	0.710523	789	7-206.13 Tracking Powders, Pest Control and Monitoring. The use of tracking powder pesticides presents the potential for the powder to be dispersed throughout t
2	0.838091	0.752362	754	Rodent bait shall be contained in a covered, tamper-resistant bait station. P FDA Food Code 2022 Chapter 8. Compliance and Enforcement Chapter 7 – 5 7-206.13 Tr
3	0.785614	0.737814	686	Also, it is easier for pests to transport the potentially toxic bait throughout the establishment. Consequently, the bait may end up on food-contact surfaces an
4	0.710526	0.581687	324	References Annex 2 – 62 7-205.11 Incidental Food Contact, Criteria. 1. Code of Federal Regulations, Title 21, Part 178.3570, Lubricants with incidental food con
5	0.709489	0.490282	457	Appl. Microbiology, 30(2):229-237. FDA Food Code 2022 Annex 2. References Annex 2 – 60 6-501.111 Controlling Pests 1. Monica Pava-Ripoll, Rachel E. Goeriz Pears

Retrieval comparison

k = 5. Cosine and recall are means over the five questions. Recomputed from raw/summary.csv.

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean cosine @ k	Recall@k	Mean retrieval latency (ms)
token-based chunking	1168	2006.1	0.574	0.542	0.8	6.8
semantic chunking	765	2758.7	0.582	0.537	1.0	6.8
sentence-window chunking	15204	977.2	0.588	0.556	0.8	10.0

One confidently scored retrieval that does not contain the answer:

For semantic chunking on Q4, rank 1 has a store score of 0.635 but a cosine similarity of only 0.372. The chunk is a bibliography fragment from the FDA Food Code annex (a CDC Emerging Infectious Diseases URL and a citation to the Public Health Service Act). It does not say whether tracking powder pesticides are allowed. The retriever likely ranked it highly because the query and the annex share pest-control vocabulary, while MiniLM matches topical neighborhood rather than the specific prohibition in section 7-206.13 which is both found by the other two techniques. The low cosine on the returned chunk is consistent with that: the vectors are only weakly aligned once the actual chunk text is re-embedded.

Observations

For Q4 query (whether the FDA Food Code allows tracking powder pesticides), sentence-window chunking did best. Rank 1 returned section 7-206.13 itself with a store score of 0.864 and cosine similarity of 0.711. Token-based also found that section, but it was buried in a lengthy chunk with 2,274 characters. Semantic chunking's rank 1 didn't even find the correct statement as it has been mentioned above. Sentence-window helps here because the prohibition is one short sentence, and indexing the sentence and attaching a three-sentence window keeps the claim intact.

The best technique is not the same on every question. In fact, sentence-window only works best on Q4. Semantic chunking is the only method with Recall@k = 1.0. Token-based and sentence-window both missed Q2, whose answer lives only in the county handout. Semantic still retrieved that handout in the top-5, at the cost of a higher average chunk length.

Conclusion

For this restaurant-inspection corpus I would use semantic chunking as the default. It is the only technique that recalled the expected source on all five questions ($\text{Recall}@k = 1.0$), it produced the fewest chunks, and a low mean retrieval latency matched token-based better than sentence-window. Sentence-window had the highest Top-1 cosine similarity and mean@k cosine similarity and won the Q4 query, so it is better when the answer is a single code sentence. However, source recall and index size outweigh that tiny cosine gain, so semantic chunking is the better overall choice.

Reproducible run instructions (also see README):

Activate the `data260` environment. Part 1 is the same FastAPI app on port 8521, now with login routes:

```
```bash
conda activate data260
python code/main.py
```
```

Open http://localhost:8521. Login is `admin` / `password`.

Retrieval-only comparison (writes `reports/hwo3/raw/summary.csv` and `METRICS.md`):

```
```bash
python code/hwo3_retrieve.py 2>&1 | tee reports/hwo3/RUN_LOG.txt
```
```

Smoke test:

```
```bash
python code/verify_hwo3.py
```
```

This writes `reports/hwo3/verification.json`.